Import Data Using Transform Maps (Spreadsheet)

Initiative: NaanMudhalvan / TamilNaduSkillDevelopmentCorporation(TNSDC)

Domain / Platform: ServiceNow Administration / Cloud Integration & Data Management

TEAM INFORMATION

Team ID	: SWTID-2026-6626
Team Size	:5
Team Leader	:JEES BALA SINGAM.K
Team Members	:JACKSON RAJ.I MATHIYAZHAGAN.R PRAVEEN.A DEENA

Executive Summary

Datafragmentationacrosslegacy formats—primarily Microsoft Excel (.xlsx) and Comma-Separated Values (.csv)—remains a significant barrier to efficient enterprise IT Service Management (ITSM). Migrating raw, unstructured, or semi-structured data directly into enterprise databases often results in data redundancy, broken foreign-key relationships, and schema corruption.

This project designs and implements an automated, secure, and fault-tolerant Extract-Transform-Load (ETL) pipeline in ServiceNow using Import Sets and Transform Maps. By establishing an intermediate staging layer (u_imp_employee_data), defining field-mapping relationships to the target production table (sys_user), enforcing unique Coalesce constraints, and executing server-side Transform Scripts (onBefore), this solution guarantees zero duplicate creation, validates incoming data integrity, and produces verifiable audit trails.

Phase 1: Ideation Phase

1.1 Problem Statement & Background

Modern enterprises frequently accumulate organizational records—such as employee onboarding data, asset allocations, and network configuration items (CIs)—across department-level spreadsheets. During system consolidation or digital transformation onto ServiceNow:

- **Manual Data Entry Bottleneck:** Transcribing thousands of records manually is labor-intensive and error-prone.
- **Data Duplication:** Without automated unique-key checks, re-importing updated spreadsheets creates duplicate target records, skewing analytics and access permissions.

- **Data Quality & Validation Deficits:** Spreadsheetslack rigid t ype -checking ,leading to missing mandatory attributes or invalid foreign references.

1.2 Proposed Solution & Objectives

An automated ETL framework built inside the ServiceNow platform:

- **Staging Layer (Import Sets):** Ingest raw spreadsheet files directly into a decoupled staging table without affecting production schemas.
- **Declarative Mapping Engine (Transform Maps):** Establish field-level associations between staging attributes and destination attributes.
- **Idempotent Upsert Logic (Coalesce):** Designate unique keys (such as user_name or email) to automatically update existing records or insert new ones.
- **Transform Scripts:** Inject server-side business rules to sanitize data, format phone numbers, and reject malformed records before database commit.

1.3 Brainstorming & Idea Prioritization

- **Manual Data Entry:** Very low speed, high failure rate — *Discarded*.
- **Direct REST API Ingestion:** High speed, high complexity — *Deferred for external systems*.
- **ServiceNow Import Sets & Transform Maps:** High speed, built-in logging and audit trail, declarative Coalesce feature — *Selected as core solution*.

Phase 2: Requirement Analysis Phase

2.1 Functional Requirements

- **FR-01:** Ingest standard .xlsx and .csvfiles up to 10,000 records per batch.
- **FR-02:** Dynamically create or map to an Import Set staging table (u_imp_*).
- **FR-03:** Map alphanumeric strings to target fields, including Reference fields (department, location) using display-value matching.
- **FR-04:** Match incoming records using one or more Coalesce fields to update existing records or insert new ones.
- **FR-05:** Reject invalid email formats or missing required fields with logged execution warnings.

2.2 Non-Functional Requirements

- **Performance:** File ingestion and transformation of 5,000 rows in under 120 seconds.
- **Security:** Role-Based Access Control restricted to import_admin and admin roles.
- **Auditability:** Record source file name, row number, execution state (Inserted, Updated, Ignored, Error), and execution timestamp.

2.3 System Specifications

Parameter	Specification
Cloud Platform	ServiceNow Utah / Vancouver / Washington DC (PDI)
Target Table	User Table (sys_user) / Configuration Item Table (cmdb_ci)
Source Datasets	Employee Master records in Microsoft Excel (.xlsx)
Scripting Engine	ServiceNow Server-Side JavaScript (ECMA-262 / Rhino engine)

Phase 3: Project Design Phase

3.1 Architectural Pipeline

> Excel / CSV Spreadsheet → ServiceNow Data Source → Import Set Staging Table (u_imp_employee_data) → Transform Map Engine (Validation Scripts, Field Mapping, Coalesce Logic) → Insert / Update → Target Production Table (sys_user)

3.2 Data Mapping Specification

Source Field (Staging)	Target Field (sys_user)	Type / Length	S et tings
u_employee_id	employee_number	String (40)	Standard Mapping
u_first_name	first_name	String (40)	Sanitized
u_last_name	last_name	String (40)	Sanitized
u_email	email	String (100)	**Coalesce: True**
u_user_name	user_name	String (40)	Mandatory Check
u_department	department	Reference	cmn_depar tment
u_active_status	active	Boolean	Target Default

Phase 4: Project Planning Phase

- **Milestone 1: Environment Setup & Data Preparation** (Excel template creation, role assignment).
- **Milestone 2: Import Set & Staging Table Definition** (Load Data module, schema creation).
- **Milestone 3: Transform Map Configuration** (Field Mapping, Mapping Assist, Coalesce setting).
- **Milestone 4: Scripting & Quality Engineering** (onBefore script implementation, validation testing).
- **Milestone 5: Documentation & Presentation** (Report compilation, presentation deck).

Phase 5: Project Development & Implementation Phase

5.1 Implementation Steps

1. Prepare and validate source spreadsheet data.
2. Load data into Import Set Table via *System Import Sets > Load Data*.
3. Create and configure Transform Map targeting sys_user.
4. Map fields using Mapping Assist and enable Coalesce on the email field.
5. Implement onBefore Transform Script for data validation.
6. Execute Transform and verify Import Set Row logs.

5.2 Server-Side Transform Script (onBefore)

```
(function runTransformScript(source, map, log, target /*null onBefore*/) {
    // 1. Mandatory Data Verification
    if (gs.nil(source.u_email) || gs.nil(source.u_user_name)) {
        ignore = true;
        log.error("Transform Rejected: Missing mandatory Email or Username on Row ID:" +
source.sys_import_row);
        return;
    }

    // 2. Email Regular Expression Validation
    var emailPattern = /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/;
    var emailString = source.u_email.toString().trim();
    if (!emailPattern.test(emailString)) {
        ignore = true;
        log.warn("Transform Ignored: Malformed email syntax detected → " + emailString);
        return;
    }

    // 3. Data Sanitization
    target.first_name = source.u_first_name.toString().trim();
    target.last_name = source.u_last_name.toString().trim();
    target.email = emailString.toLowerCase();
})(source, map, log, target);
```

Phase 6: Project Documentation & Testing Phase

6.1 Testing Matrix

Test Case ID	Test Scenario	Expected Outcome	Result
TC-01	Standard New Record Insertion	Target record created in sys_user	PASS
TC-02	Coalesce-Driven Record Update	Existing record updated, no duplicate created	PASS
TC-03	Missing Mandatory Field Filter	Row ignored, error logged in execution history	PASS
TC-04	Invalid Email Format Rejection	Malformed row ignored with script warning	PASS
TC-05	Reference Field Resolution	Department matched correctly via display value	PASS

6.2 Conclusion & Future Scope

Conclusion: Theimplementationsuccessfully provides zero-duplicate bulk ingestion while enforcing data integrity and platform security standards.

Future Scope: Automated scheduled imports via SFTP/MID Server integration, and automated summary email notifications sent to administrators upon run completion.